

Physical AI

Fall 2025

Course Project Specification

Deadline: 2026/01/01 23:59

Overview

This semester, the course project focuses on building an intelligent robotic manipulation system using the UMI device [1] and the Isaac Sim simulator [2]. You will first collect real-world demonstration data and then reconstruct these demonstrations in simulation to create training datasets. Next, you will train a manipulation policy and evaluate its performance across different simulated environment layouts. The goal is to develop a policy that can reliably perform pick-and-place operations in the simulated ED305 scene, even under varying object configurations. Through this project, you will gain hands-on experience with the complete robot learning pipeline, from data collection, training, and deployment.

Pipeline

STEP1. Real-World Data Collection with UMI

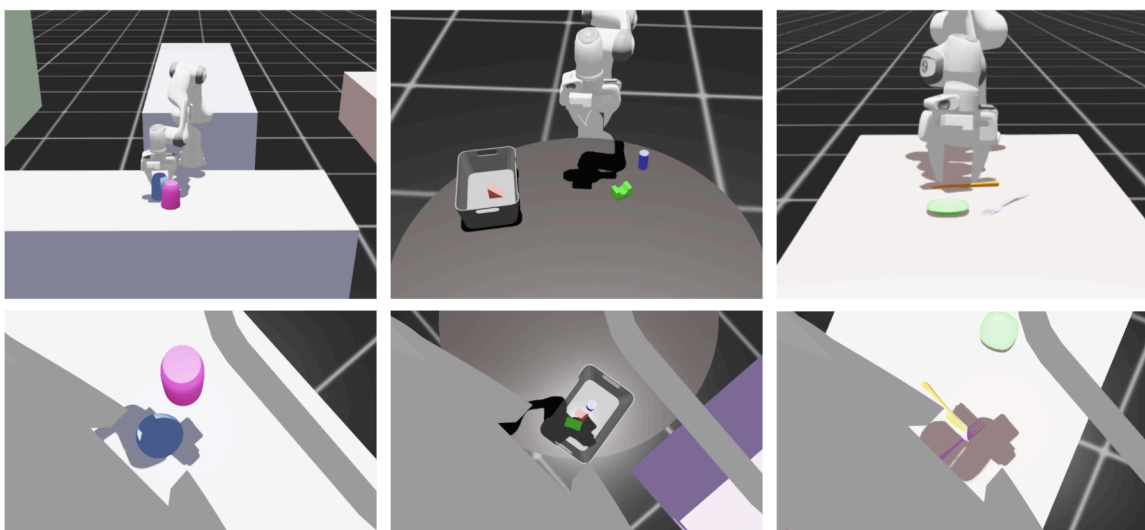
Using the UMI [1] devices that you assembled during the course, collect real-world demonstration data in the ED305 classroom. Each group is assigned one of the following three pick-and-place tasks:

- **Cup Stacking in the Kitchen:** Randomly place the blue and pink cups upside down on the countertop. Then, pick up the blue cup and stack it on top of the pink cup.
- **Cutlery Arrangement on the Dining Table:** Randomly place the fork and knife on the table. Then, place the knife on the right side of the plate and the fork on the left side of the plate.
- **Toy Blocks Collection in the Living Room:** Randomly place three toy blocks on the tabletop. Then, collect all the scattered blocks and place them into the white basket.



STEP2. Simulation Reconstruction and Data Re-Collection

Since direct operation of a real robotic arm in the ED305 classroom is currently not available, we provide a corresponding virtual environment in NVIDIA Isaac Sim [2]. In this step, you will run the scripts we provide to reproduce real-world demonstrations within the simulation. This process automatically collects the corresponding observations and robot end-effector trajectories in the virtual environment, which will serve as the training data for subsequent model learning.



Due to some limitations encountered during implementation, the reproduced demonstrations in simulation **only preserve the object poses from the original recordings, rather than the full robot motion trajectories**. Instead, we employ a naive rule-based motion planning method to generate feasible actions for task execution. In addition, due to unexpected issues that prevent the robotic arm from reliably grasping objects, we adopt a simplified grasping approximation, where objects are “attached” to the robot end-effector and move together with it. It is also worth noting that certain object properties (e.g., object size and geometry) and environment configurations in simulation may not perfectly match those in the real world. These discrepancies are introduced as practical engineering adjustments to ensure the overall system remains functional and the data collection pipeline can operate reliably. You may also observe that the reconstructed object poses occasionally differ from those obtained in your own collected data. Such

discrepancies can arise from detection and coordinate transformation errors, as well as physical interactions and collisions within the simulated environment. Nevertheless, this data collection pipeline is designed to record a demonstration only when the task success criteria are satisfied, ensuring the overall validity of the collected dataset.

STEP3. Model Training with Diffusion Policy

After collecting the simulated dataset, the next step is to train a model to perform the corresponding pick-and-place tasks. We adopt the Diffusion Policy [3], a recently popular imitation learning approach in robotic learning. Given the environment observations and the robot state, the model learns to generate the corresponding robot end-effector trajectories. In this stage, the collected dataset is split into training and validation sets, and evaluates the training performance in an offline manner.

STEP4. Inference and Evaluation

During inference, the trained policy is deployed in the simulation and evaluated online under different environment layouts to assess its capability, with a particular focus on robustness to varying object placements. **The scripts used for this evaluation will not be released. Instead, we will perform the evaluation using the trained diffusion policy model weights submitted by each group.** Model performance will be ranked separately for each task. The group achieving the best performance on each task will receive additional bonus points on the final total course score.

Final Notes & Tips

We encourage you to explore, experiment, and have fun throughout this project. This pipeline is designed to help you gain hands-on experience with the full robotic learning workflow, from data collection to model training and evaluation. Before you start, here are two important tips to keep in mind:

- **Data quality matters.** The performance of your model largely depends on the data you collect. Think carefully about the final task requirements and evaluation settings, and design your data accordingly.
 - Although collecting additional real-world data is not allowed, you can modify the data generation scripts to obtain more training data. Just make sure to clearly describe any changes you make in both the presentation and the report. Possible aspects you can adjust include, but are not limited to, the following:
 - **Solve real-world UMI data processing issue:** As you may have experienced previously, UMI data conversion is not always guaranteed to succeed for every recording. You are expected to

analyze the data processing pipeline and identify potential reasons that could cause these conversion failures, and if there is a possibility to resolve them, because the simulation data collection is highly dependent on real-world collected data.

- **Improve simulation data collection:** You may modify the predefined pipeline in the simulation data collection stage, for example, (1) improve pick-and-place strategy by introducing multiple grasp poses or adding randomized motion to the robot trajectory, (2) change camera pose (default is eye-in-hand) to have better view of environments.
- **Build up custom data collection methods:** You may implement your own data collection interfaces or pipelines in simulation to generate additional training data. For example, you may: (1) introduce keyboard-based control for manual data collection, or (2) design a retry mechanism that allows multiple attempts within each episode by applying different modifications until data is successfully collected.
- **Hyperparameters are crucial.** Model performance is highly sensitive to hyperparameter choices. In the beginning, you can use the default parameters for training, but we strongly recommend referring to the original Diffusion Policy [3] paper for detailed discussions and insights. You will likely gain valuable understanding from it.
 - In particular, some hyperparameters are closely related to task horizon length and multi-step behaviors, so we encourage you to experiment with different parameter combinations.

=====

The aim of this project is to help you understand the overall robot learning pipeline. Therefore, if you are unable to make significant modifications to the provided code and pipeline, do not worry. You can still go through the entire project workflow, reflect on the challenges and issues you encountered during implementation, analyze their possible causes, and propose potential solutions. Doing so will still allow you to achieve a solid score in this project.

=====

Project Setup and Guidelines

We have prepared the following environments and platforms to support you in completing this final project:

- **Voilab Repository ([code](#) / [usage guide](#)):** This github repository contains all the necessary code for the project, including conversion of UMI-collected datasets, virtual environment setup, data collection, model training, and evaluation. **Please make sure to pull the latest version, as the repository has been updated.**
- **Glows.ai ([usage guide](#)):** This platform provides GPU resources for model training and experimentation.
- **Final Project QA Page ([link](#)):** If you have any questions about the final project, feel free to ask questions on the notion page.

Submission

The deadline for submission of the project is **01/01 (Tue.) at 23:59**. You need to submit the following files to the E3 system:

└ 📁 Team{TEAM_ID}_project_link.txt

The **project link** should be a **folder**, which includes:

- **Presentation Video:** Video named Team{TEAM_ID}_presentation.mp4, in which you briefly explain your project work within **5 minutes**.
- **Project Report:** Report named Team{TEAM_ID}_project_report.pdf, providing a detailed description of your implementation, design choices, and insights throughout the project. Please follow the guidelines described in the “Report Format” section below.
- **models/:** A folder named models/ that contains **three** subfolders (with arbitrary names). Each subfolder corresponds to one trained model checkpoint and config file. We will evaluate all three submitted models and select the one with the best performance as your final score.
 - **Model Checkpoint:** The trained model checkpoint file.
 - **Model Config File (.yaml):** The configuration file used for building up model architecture and hyperparameters for training and evaluation.
- **Others:** Any other files if needed.
- **Readme.txt:** Clearly specify how to run your code when additional files are required. If no additional files are needed, this should also be explicitly stated.

Notice

- **Only one team member needs to submit the link.txt file to the E3 system.**

- All submission files must be uploaded to the Google Drive link provided. Once the deadline has passed, no further modifications are allowed. If any file is found to have a modification timestamp later than the deadline, **it will be treated as a late submission and penalized accordingly.**
- If any file within the submitted link cannot be opened or downloaded properly, the part of submission will not be graded.
- For this final project, we **DO NOT accept any late submissions.** A one-day late submission will result in a **20-point deduction** from the final grade.
- Any formatting errors will receive a **10-point deduction.**

Grading Policy

The grading policy of this course is below:

- **Homework (60%)**
 - HW1 (20%)
 - HW2 (20%)
 - HW3 (20%)
- **Final Project (40%)**
 - Presentation Video (20%)
 - Project Report (20%)
- **(Bonus)** For the final project, teams with the best performance among all groups in each task will receive a **+5 bonus points added to the total course score.**

Presentation Video Format (20%)

There is no fixed pipeline that you must follow exactly; however, your presentation should include the following three aspects:

- Task description
- What modifications you made to the provided pipelines or code
- What you learned from this project

Report Format (20%)

The project report requires more detailed explanations and should therefore follow the format outlined below:

1. Task Description

- Briefly introduce the task you are working on.

2. Data Collection

- **Real-World Data Collection with UMI**
 - Explain how you collected the data using the UMI device.
 - Describe how the collected demonstrations were converted into a training dataset.
 - Provide visualizations of the collected data.
- **Simulation Data Collection in Isaac Sim**
 - Describe dataset types and size.
 - (Optional) If you modified the data collection code or procedures, clearly describe the changes in this section.
 - Provide visualizations of the collected data.

3. Model Training

- **Model Architecture**
 - Describe the model architecture used in your project.
 - (Optional) If you modified the original architecture, explain the changes and your motivations.
- **Training Procedure**
 - Describe the overall training procedure.
 - List and explain the key hyperparameters.
 - (Optional) If you adopted a different training procedure, describe your approach and the reasons for your design choices in this section.

4. Results

Since the evaluation scripts will not be released, we recommend splitting your reconstructed dataset into training and validation sets. Validation performance can serve as one of the criteria for selecting and comparing models during development.

- **Quantitative Result and Discussion**
 - Include training and validation loss curves to illustrate the learning behavior of your model.
 - Describe and analyze your results, including performance trends and convergence behavior.
 - (Optional) If you implement the online evaluation by yourself, you can also describe and show the results here.

6. Discussion

Share your observations, challenges encountered during the project, and lessons learned. You may also discuss potential improvements or future directions.

7. Work Distribution

- Each team member's assigned tasks and responsibilities.

- Each team member's percentage of work contribution.

Reference

- [1] Chi, Cheng, et al. "Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots." RSS (2024). Project page: <https://umi-gripper.github.io/>
- [2] Isaac Sim Document: <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html>
- [3] Chi, Cheng, et al. "Diffusion policy: Visuomotor policy learning via action diffusion." The International Journal of Robotics Research 44.10-11 (2025): 1684-1704. Project Page: <https://diffusion-policy.cs.columbia.edu/>